

Rapport
Projet UL2IN013 :
Système de Simulation
et
Contrôle de Robot
DEXTER

GROUPE : "BoysBand"

SOMMAIRE

- I Introduction
- II Mode d'Emploi et Exécution
- III Architecture et Fonctionnement
- IV Modèle Physique et Cinématique
- V Conclusion

Mateusz | Alex | Rayane | Xiangkun | Kevin

ANNÉE 2025/2026

1° INTRODUCTION

Le projet "Robot Dexter" consiste à réaliser un système simple de pilotage d'un robot à deux roues motrices, implémentant un environnement de simulation pour le robot. L'objectif est de pouvoir tester les comportements du robot en simulation, qui profite d'une flexibilité des situations et configurations du robot afin de pouvoir réutiliser la même logique de commande sur la plateforme physique, assurant une efficacité temporelle et matérielle pour le développement de l'application.

2° MODE D'EMPLOI ET EXÉCUTION

Prérequis

- [Python 3.9+](#)
- Installer **Pygame** pour la simulation.
- Accéder au github du projet : <https://github.com/CrazyDreamer2000/ProjetRobotique-UL2IN013>, puis dans la branche "Branche-Dev", et télécharger le code en cliquant sur "Download ZIP".

Lancement du Programme

Dans un terminal, se placer dans le répertoire **ProjetRobotique-UL2IN013/BoysBand** :

```
>>> cd .\ProjetRobotique-UL2IN013\BoysBand\
```

Pour lancer le programme, entrez **python main.py**. Vous pouvez rajouter des paramètres (voir Paramètres disponibles).

```
>>> python main.py
```

```
>>> python main.py --simu --algo 'polygone' --vitesse_roues 3.0
```

Déroulement de l'Exécution

Au démarrage, `main.py` initialise les composants dans l'ordre suivant :

1. Création du monde de simulation (**Monde**).
2. Création du traducteur adapté (**TraducteurSimu** ou **TraducteurReel**).
3. Lancement du thread d'affichage (Pygame).
4. Instanciation de l'algorithme de contrôle choisi (**AlgoBase**).
5. Exécution de la boucle principale.

Paramètres disponibles (lus par `parser.py`) :

- `--simu` : Exécute le programme en mode simulation (robot réel par défaut sinon)
- `--algo [nom_algo]` : Définit la stratégie de mouvement. Options disponibles : `polygone`, `avancer`, `foncerdevant`.
- `--vitesse_roues [v]` : Règle la vitesse de commande maximale des roues (exprimée en rad/s).

3° ARCHITECTURE ET FONCTIONNEMENT

Cette section présente la logique globale du système et les interactions entre blocs.

Le système est structuré en modules pour une logique claire et des interactions définies :

- **main.py** : Lancement et boucle principale.
- **controle/** : Algorithmes de contrôle (primitives, compositions, stratégies).
- **traducteur.py** : Adapter pour lier commandes du Contrôleur à l'exécution (simu/réal).
- **monde/** : Gestion de la physique et des capteurs en simulation.
- **core/** : Services bas niveau (types, calculs) et gestion du robot simulé.
- **affichage/** : Visualisation graphique (Pygame).

L'**Adapter Pattern** (Traducteur) assure la portabilité : **TraducteurSimu** vers **Monde** ou **TraducteurRéal** vers **API Robot** (Robot2IN013). Les mêmes algorithmes de mouvement sont utilisés dans les deux modes.

Les **algorithmes de contrôle** sont modulaires et basés sur des primitives (ex: tourner, avancer).

Les composants principaux (*main.py*) et d'affichage (*Affichage.py*) utilisent des **threads** distincts. Un verrou **RLock** est utilisé pour la **synchronisation** des données partagées (Monde, robot, obstacles) et la prévention des *race conditions*.

Représentation des classes (Schéma UML)

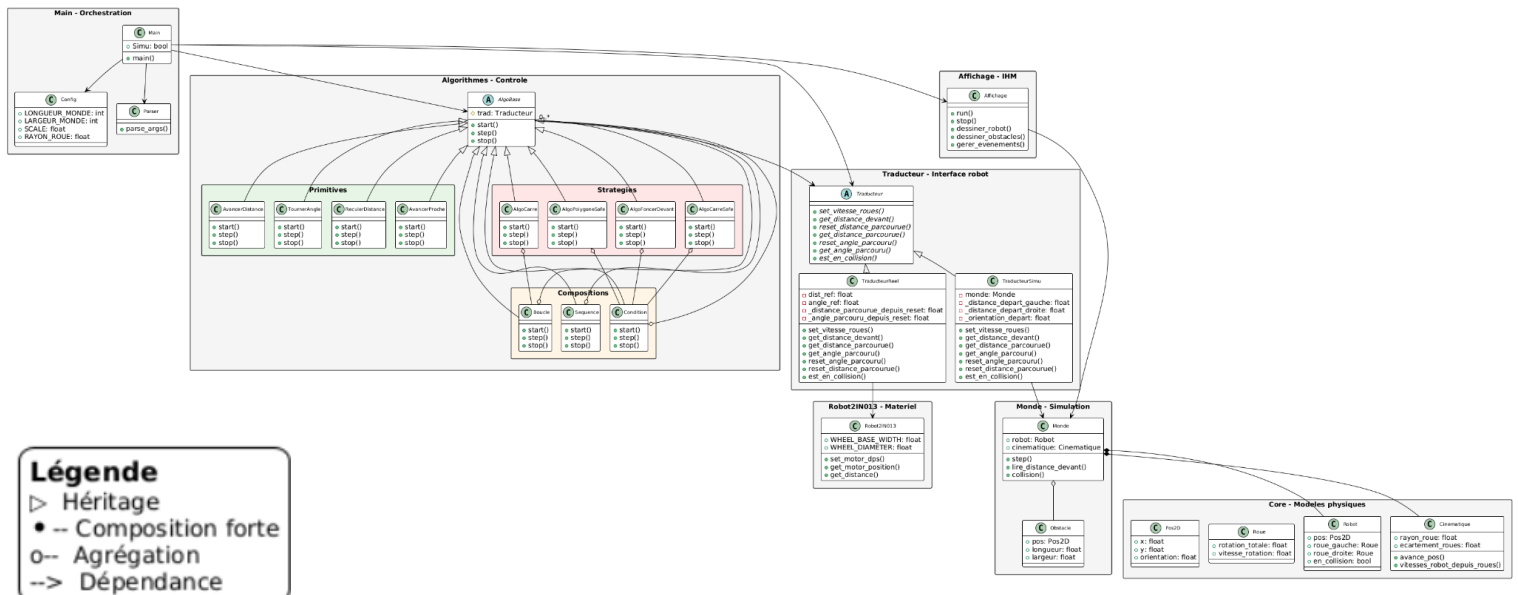
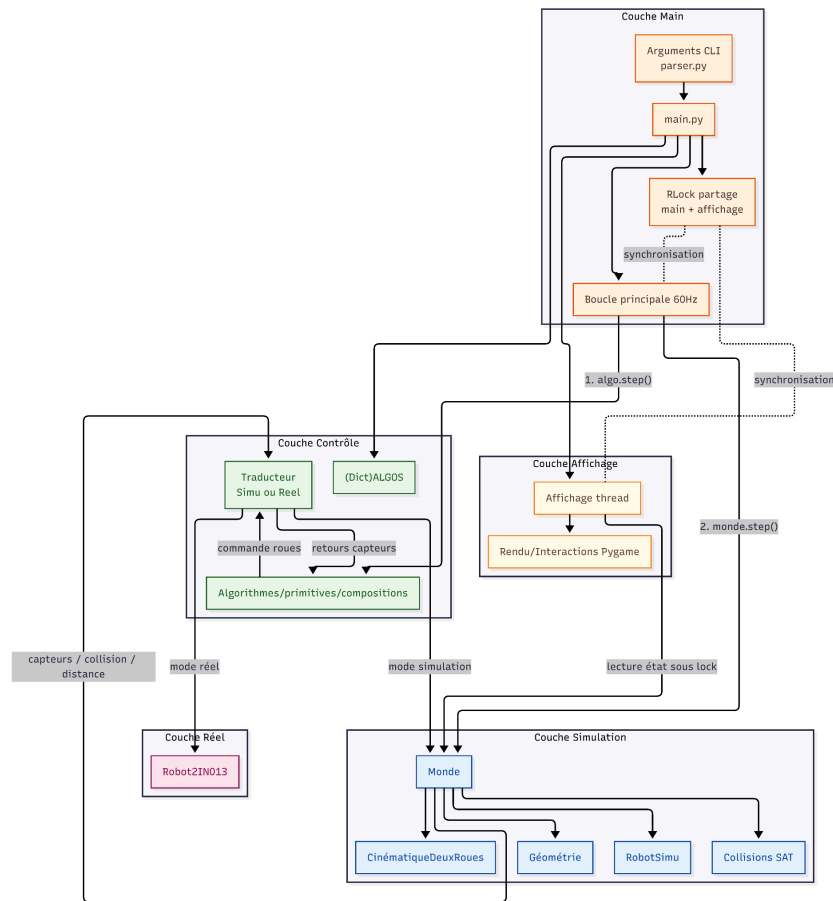


Schéma fonctionnel du projet



4° MODÈLE PHYSIQUE ET CINÉMATIQUE

Cette partie détaille les hypothèses physiques du robot et les équations utilisées pour simuler son mouvement.

1. Paramètres Physiques

Les calculs utilisent les constantes définies dans la configuration du robot :

- R : Rayon des roues (33.25 mm, soit 66.5/2).
- L : Écartement entre les roues (Entraxe = 117 mm).

2. Calcul des Vitesses (Modèle Différentiel)

À partir des vitesses de rotation de la roue gauche (ω_g) et de la roue droite (ω_d) en rad/s, on calcule la vitesse linéaire de chaque roue :

La vitesse linéaire du centre du robot (v) et sa vitesse angulaire (ω) :

- $v_g = \omega_g \times R$
- $v_d = \omega_d \times R$

- $v = (v_g + v_d) / 2$
- $\omega = (v_d - v_g) / L$

3. Intégration de la Posture (Mise à jour de la position)

À chaque pas de temps dt , la posture du robot (définie par sa position (x,y) et son orientation θ) est mise à jour localement au sein du moteur de simulation. Afin de garantir une meilleure fidélité physique, le robot calcule sa nouvelle position en distinguant deux types de trajectoires :

Cas 1 : Mouvement quasi-rectiligne

Si la vitesse de rotation est négligeable ($|\omega| < 10^{-9}$), on utilise une approximation linéaire pour éviter une division par zéro :

- $x_{t+dt} = x_t + v \cdot \cos(\theta_t) \cdot dt$
- $y_{t+dt} = y_t + v \cdot \sin(\theta_t) \cdot dt$
- $\theta_{t+dt} = \theta_t$

Cas 2 : Mouvement en arc de cercle

Pour tout mouvement combinant translation et rotation, le robot suit un arc de cercle parfait de rayon $R=v/\omega$. Les nouvelles coordonnées sont alors :

- $\theta t+dt = \text{normaliser}(\theta t + \omega \cdot dt)$
- $x t+dt = x t + \omega v \cdot (\sin(\theta t+dt) - \sin(\theta t))$
- $y t+dt = y t - \omega v \cdot (\cos(\theta t+dt) - \cos(\theta t))$

4. Transformation de Repère (Local vers Monde)

Pour l'affichage et la détection de collisions (SAT), les points du robot définis dans son repère local sont transformés en coordonnées "monde" :

- $x_{\text{monde}} = x_{\text{local}} \cdot \cos(\theta) - y_{\text{local}} \cdot \sin(\theta) + x_{\text{robot}}$
- $y_{\text{monde}} = x_{\text{local}} \cdot \sin(\theta) + y_{\text{local}} \cdot \cos(\theta) + y_{\text{robot}}$

5. Gestion des Angles

L'orientation 'a' du robot est systématiquement ramenée dans l'intervalle $[-\pi, \pi]$ pour éviter les dérives de calcul via la fonction de normalisation :

- $\text{Angle_Normalisé} = (a + \pi) \pmod{2\pi} - \pi$

Simulation de l'Environnement et Détection de Collision

Le module **monde/monde.py** est responsable de la mise à jour de l'environnement :

1. La **création aléatoire d'obstacles** (polygones convexes).
2. La **vérification des collisions** (robot, obstacles, limites du monde) à chaque cycle.
3. La **simulation des capteurs** (estimation de la distance frontale, mise à jour des états).

La **Détection de Collision** est implémentée dans monde/collisions.py et utilise le **SAT (Separating Axis Theorem)**. En cas de collision détectée à la prochaine position, le mouvement est annulé et l'état **en_collision** est activé pour informer le Contrôleur.

5° CONCLUSION

Notre projet se conclut par la mise en place réussie d'un système de pilotage simple et modulaire pour le robot mobile **DEXTER**. Grâce à une passerelle (Traducteur) reliant la simulation au robot réel, nous pouvons appliquer la même logique de contrôle sans aucune modification logicielle.

L'architecture repose sur une **boucle itérative efficace** : lecture des capteurs, calcul des commandes par le **Contrôleur** (*primitives, compositions, stratégies*), envoi de ces ordres via le **Traducteur** et affichage en temps réel. Le simulateur intègre un modèle cinématique, une détection de collisions (**SAT**) ainsi qu'une interface graphique développée avec **Pygame**. Les tests ont démontré que le robot est capable de tracer un polygone, d'éviter des obstacles ou de s'arrêter avant une collision frontale. Cependant, le suivi de balise n'a pu être finalisé en raison d'un dysfonctionnement de la caméra dans la classe Python du robot.

Ce projet n'étant pas exhaustif, plusieurs *pistes d'amélioration* sont envisageables :

- **Interface de configuration ergonomique** : Développer une interface graphique (GUI) intuitive pour la simulation afin de s'affranchir du terminal. L'ajout d'icônes permettrait de placer dynamiquement des obstacles à la souris, de sélectionner l'algorithme actif et de régler la vitesse du robot.
- **Robustesse du code** : Implémenter une gestion plus rigoureuse des exceptions pour sécuriser l'exécution.
- **Aide au débogage** : Peaufiner le mode "Debug" pour faciliter l'analyse des flux de données.
- **Restructuration des modules** : Réorganiser la racine du projet pour ne laisser que le fichier **main.py** en point d'entrée, afin d'optimiser la clarté de l'arborescence.